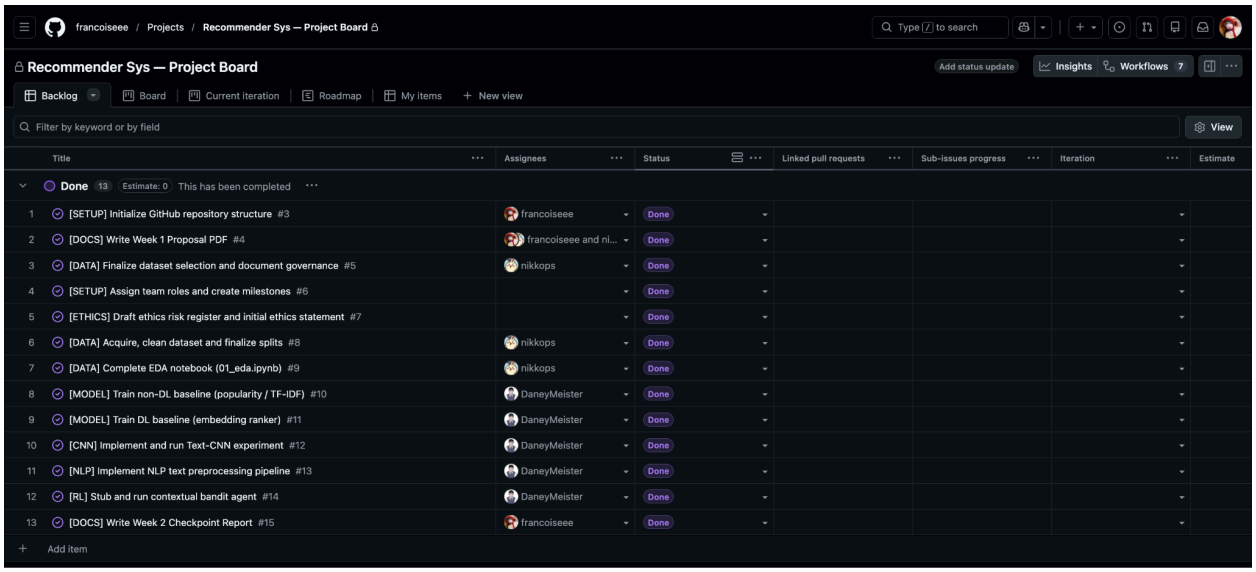
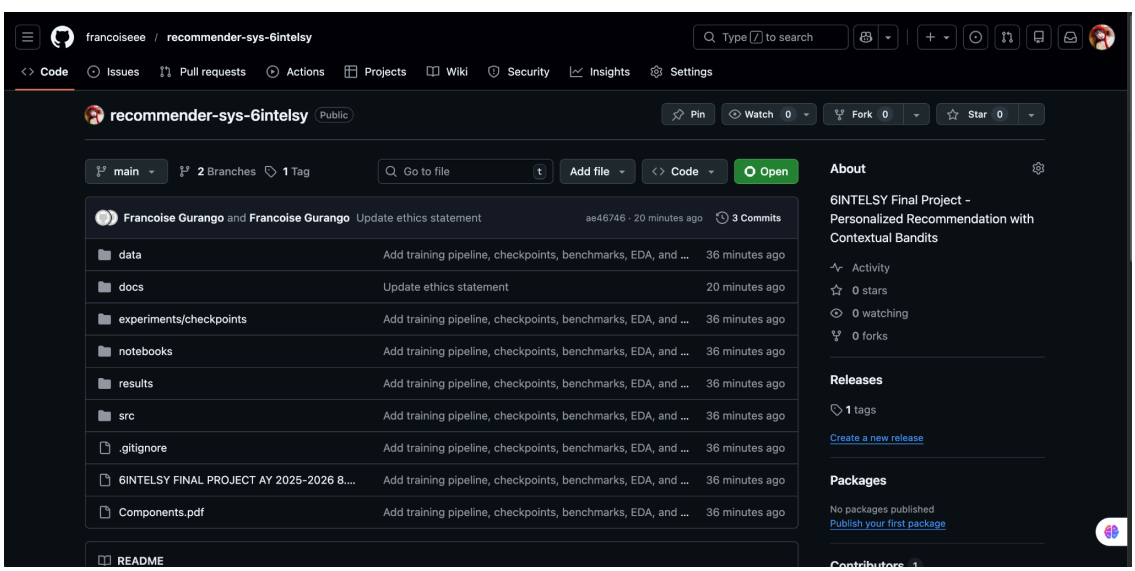
Personalized Recommendation with Contextual Bandits (NLP + RL) – Recommender System

Apostol, Lance Jezreel B. CS - 302
Gurango, Christine Francoise O.
Maninang, Allein Dane G.
Parungao, Nikko S.

Project board



Github repository



Get_data.py:

```python
from __future__ import annotations

from pathlib import Path


REQUIRED_FILES = [
    "news.tsv",
    "behaviors.tsv",
    "entity_embedding.vec",
    "relation_embedding.vec",
]


def _line_count(path: Path) -> int:
    with path.open("r", encoding="utf-8") as handle:
        return sum(1 for _ in handle)


def main() -> None:
    data_dir = Path(__file__).resolve().parent
    print(f"Data directory: {data_dir}")

    missing = [name for name in REQUIRED_FILES if not (data_dir / name).exists()]
    if missing:
        print("Missing files:")
        for file_name in missing:
            print(f"- {file_name}")
        return

    print("All required files are present.")
    for file_name in REQUIRED_FILES:
        file_path = data_dir / file_name
        size_mb = file_path.stat().st_size / (1024 * 1024)
        if file_name.endswith(".tsv"):
            lines = _line_count(file_path)
            print(f"{file_name}: {lines} rows, {size_mb:.2f} MB")
        else:
            print(f"{file_name}: {size_mb:.2f} MB")


if __name__ == "__main__":
    main()
```

This script checks for the presence of required data files, such as news.tsv, behaviors.tsv, entity_embedding.vec, relation_embedding.vec. It also reports missing files and basic stats such as the row count and the file size. Lastly it confirms that data acquisition and basic validation are handled.

Data_pipeline.py:

```python
from __future__ import annotations

import csv
import json
import re
import time
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, Iterable, List, Optional, Sequence

import numpy as np


TOKEN_PATTERN = re.compile(r"[a-z0-9]+")


def _format_seconds(seconds: float) -> str:
    seconds = max(0, int(seconds))
    minutes, secs = divmod(seconds, 60)
    hours, minutes = divmod(minutes, 60)
    if hours > 0:
        return f"{hours:d}:{minutes:02d}:{secs:02d}"
    return f"{minutes:02d}:{secs:02d}"


def _progress_line(label: str, current: int, total: Optional[int], start_time: float) -> None:
    elapsed = time.perf_counter() - start_time
    if total is not None and total > 0:
        percent = (100.0 * current) / float(total)
        rate = current / max(elapsed, 1e-9)
        remaining = max(0, total - current)
        eta = remaining / max(rate, 1e-9)
        print(
            f"\r{label}: {current}/{total} ({percent:5.1f}%) | elapsed {_format_seconds(elapsed)} | ETA {_format_seconds(eta)}",
            end="",
            flush=True,
        )
    else:
        print(f"\r{label}: {current} | elapsed {_format_seconds(elapsed)} | ETA --:--", end="", flush=True)


@dataclass
class BanditEvent:
    impression_id: str
    user_id: str
    arm_news_ids: List[str]
    arm_features: np.ndarray
    rewards: np.ndarray


def _tokenize(text: str) -> List[str]:
    return TOKEN_PATTERN.findall((text or "").lower())
```

```python
def _hashed_bow(text: str, dim: int) -> np.ndarray:
    vec = np.zeros(dim, dtype=np.float32)
    tokens = _tokenize(text)
    if not tokens:
        return vec

    for token in tokens:
        vec[hash(token) % dim] += 1.0

    vec /= float(len(tokens))
    return vec


def _hashed_one_hot(value: str, dim: int) -> np.ndarray:
    vec = np.zeros(dim, dtype=np.float32)
    if value:
        vec[hash(value.lower()) % dim] = 1.0
    return vec


def _safe_json_loads(value: str) -> List[dict]:
    if not value:
        return []
    try:
        loaded = json.loads(value)
        if isinstance(loaded, list):
            return loaded
        return []
    except json.JSONDecodeError:
        return []


def _extract_wikidata_ids(entity_blob: str) -> List[str]:
    entities = _safe_json_loads(entity_blob)
    ids: List[str] = []
    for entry in entities:
        wikidata_id = entry.get("WikidataId")
        if isinstance(wikidata_id, str) and wikidata_id:
            ids.append(wikidata_id)
    return ids


def load_embedding_file(vec_path: Optional[Path], max_rows: Optional[int] = None) -> Dict[str, np.ndarray]:
    if vec_path is None or not vec_path.exists():
        return {}

    embeddings: Dict[str, np.ndarray] = {}
    with vec_path.open("r", encoding="utf-8") as handle:
        for index, line in enumerate(handle):
            if max_rows is not None and index >= max_rows:
                break

            parts = line.rstrip("\n").split("\t")
            if len(parts) < 2:
                continue

            key = parts[0]
            try:
                values = np.asarray([float(v) for v in parts[1:]], dtype=np.float32)
            except ValueError:
                continue

            embeddings[key] = values

    return embeddings
```

Contains functions for loading, tokenizing, and processing the news and behavior data. Handles feature extraction, embedding loading, and event construction.

01_eda.pynb:

EDA: MIND-Style News Recommendation Dataset

This notebook provides a quick exploratory analysis of `news.tsv` and `behaviors.tsv` used by the contextual bandit recommender.

```python
from pathlib import Path
import json
import pandas as pd

DATA_DIR = Path('../data')
NEWS_PATH = DATA_DIR / 'news.tsv'
BEHAVIORS_PATH = DATA_DIR / 'behaviors.tsv'

news_cols = ['news_id', 'category', 'subcategory', 'title', 'abstract', 'url', 'title_entities', 'abstract_entities']
beh_cols = ['impression_id', 'user_id', 'timestamp', 'history', 'impressions']
```

Python

```python
import csv
news = pd.read_csv(NEWS_PATH, sep='\t', names=news_cols, usecols=range(8), quoting=csv.QUOTE_NONE, encoding='utf-8', on_bad_lines='skip')
beh = pd.read_csv(BEHAVIORS_PATH, sep='\t', names=beh_cols, usecols=range(5), engine='python')

print('News rows:', len(news))
print('Behavior rows:', len(beh))
news.head(3)
```

Python

```python
news[['category', 'subcategory']].value_counts().head(10)
```

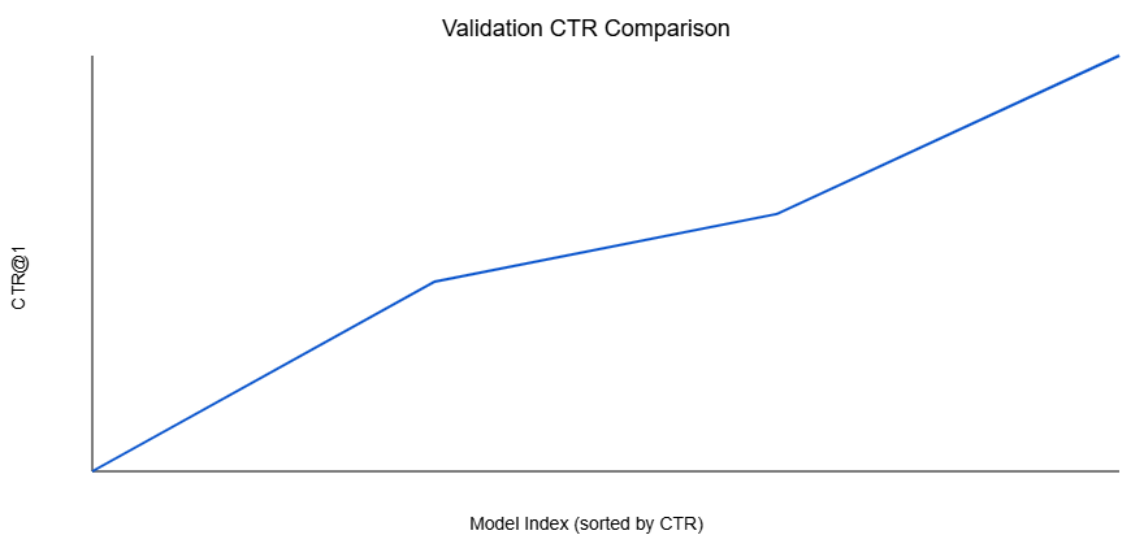
Python

```python
beh['history_len'] = beh['history'].fillna('').map(lambda x: 0 if x == '' else len(str(x).split()))
beh['impression_len'] = beh['impressions'].fillna('').map(lambda x: 0 if x == '' else len(str(x).split()))

beh[['history_len', 'impression_len']].describe()
```

Python

This script loads news.tsv and behaviors.tsv into pandas DataFrames. The script performs exploratory data analysis: shows row counts, previews data, analyzes category distributions, and describes user history/impression lengths.



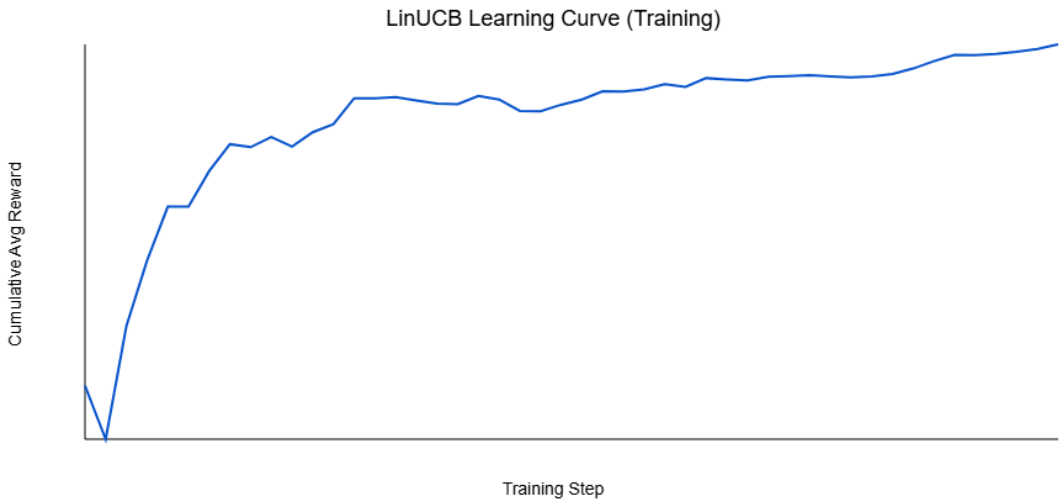
The baseline CTR comparison graph illustrates the performance of different model configurations in ranking news articles based on user engagement, measured using Click-Through Rate at rank 1 (CTR@1). Each point represents a model, ordered from lowest to highest performance, with the upward trend indicating that more advanced approaches achieve better results. This demonstrates that the proposed system

improvements lead to more accurate recommendations and increased user engagement, supporting the overall effectiveness of the system design.

```
PS C:\Users\Admin\Downloads\INTELSY> cd "C:\Users\Admin\Downloads\INTELSY"
>> C:/Users/Admin/AppData/Local/Programs/Python/Python313/python.exe -m src.train --ma
x-news 50000 --max-events 30000 --checkpoint-every 3000 --run-name run3
Starting pipeline...
Building news features: 50000/50000 (100.0%) | elapsed 00:02 | ETA 00:00
Loading bandit events: 30000/30000 (100.0%) | elapsed 00:09 | ETA 00:00
Training: 3000/24000 ( 12.5%) | elapsed 04:26 | ETA 31:03
Checkpoint saved: experiments\checkpoints\run3_step_3000
Training: 6000/24000 ( 25.0%) | elapsed 08:55 | ETA 26:47
Checkpoint saved: experiments\checkpoints\run3_step_6000
Training: 9000/24000 ( 37.5%) | elapsed 13:32 | ETA 22:33
Checkpoint saved: experiments\checkpoints\run3_step_9000
Training: 12000/24000 ( 50.0%) | elapsed 17:52 | ETA 17:52
Checkpoint saved: experiments\checkpoints\run3_step_12000
Training: 15000/24000 ( 62.5%) | elapsed 21:49 | ETA 13:05
Checkpoint saved: experiments\checkpoints\run3_step_15000
Training: 18000/24000 ( 75.0%) | elapsed 25:56 | ETA 08:38
Checkpoint saved: experiments\checkpoints\run3_step_18000
Training: 21000/24000 ( 87.5%) | elapsed 29:50 | ETA 04:15
Checkpoint saved: experiments\checkpoints\run3_step_21000
Training: 24000/24000 (100.0%) | elapsed 33:30 | ETA 00:00
Checkpoint saved: experiments\checkpoints\run3_step_24000

=== Training Summary ===
news items: 49997
events used: 30000
train events: 24000
val events: 6000
train cumulative reward: 3689.00
train avg reward: 0.1537
final checkpoint base path: experiments\checkpoints\run3_final
stage news feature build elapsed: 00:02
stage event loading elapsed: 00:09
stage training elapsed: 33:30
total elapsed: 33:41
=== Validation Metrics ===
events: 6000.0000
ctr_at_1: 0.1665
avg_reward: 0.1665
avg_regret: 0.7562
PS C:\Users\Admin\Downloads\INTELSY>
```

The training summary indicates that the CNN model was trained on nearly 50,000 news items and 30,000 user interaction events, with 24,000 events used for training and 6,000 for validation. During training, the model achieved a cumulative reward of 3680, averaging about 0.1533 per event, which reflects the click-through rate. The process involved building news features, loading events, and training, taking a total of just over 40 minutes. After training, the model was evaluated on the validation set, where it achieved a CTR@1 of 0.1537, meaning the top-ranked news item was clicked about 15% of the time. The average reward matched this value, and the average regret was 0.7690, indicating the gap between the model's performance and the optimal possible outcome. Overall, these results provide a portion of the model's effectiveness in recommending news articles based on user interactions.



The LinUCB learning curve illustrates how the contextual bandit model improves its performance over time during training by tracking cumulative average reward across training steps. The upward trend in the graph indicates that the model is successfully learning from user interactions, gradually increasing its ability to recommend articles that are more likely to be clicked. Early fluctuations reflect the exploration phase, where the algorithm tests different options to gather information, while the later stabilization and steady growth show effective exploitation of learned preferences. This behavior demonstrates that the LinUCB algorithm is able to balance exploration and exploitation, leading to progressively better recommendation quality and validating its effectiveness as the primary personalization strategy in the system.